Contrastive Learning solution for Table Version Search

Project for Data Lake Management course

Alessandro Di Girolamo

Project repository: https://github.com/aledigirm3/advanced-topics-computer-science/tree/main/DataLake-Management

Keywords: Sentence Embedder, Data Lake Management, Compuret Science, Table Version, Version Search, Contrastive Learning, Neural Network

1. Introduction

The following project aims to experiment with a solution for searching different versions of tables within a data lake.
Users are often interested in querying data from tables and would ideally want the result to be the version most relevant to their query.
In the context of data lake management, this is both a critical and challenging task.
The report outlines the various steps to follow in order to replicate the experiment.

2. Problem Analysis

Searching for versions of a table within a data lake containing tables from different domains is a complex task that can be approached using various solutions.
There are several factors to consider when tackling this problem, such as the number of tables present, the size of these tables, and, most importantly, the fact that these tables, even though belonging to different domains, may have attributes with identical (or similar) names. This could potentially confuse the algorithms or models used to address the task.
For the following experiment, the tables from *autopipeline-benchmarks* were used.

To replicate the experiment, create a folder named 'tables' inside DataLake-Management and import (insite 'tables') the *commercial-pipelines* and *github-pipelines* folders from the *autopipeline-benchmarks* repository.

3. Methodology

The following section outlines the solution approach for the version search task, which focuses on a methodology based on contrastive learning.

3.1 Dataset Preparation

The dataset was created using the tables from autopipeline-benchmarks (as described in the previous section), which were preprocessed.
After reorganizing the data, the tables were taken and possible pairs were created, both positive and negative (label 1 and label 0). These pairs were then embedded using the *all-MiniLM-L6-v2* model (from sentence-transformer), and subsequently saved using the Python pickle library. This library allows binary serialization of data, enabling faster reading compared to CSV files. Each embedding includes all columns of the table, with the first 10 rows from each column (in an attempt to capture the structure and context of the table). This procedure is performed for both the test and training datasets.

To replicate this step of the experiment, after importing the folders as described at the end of the previous section, simply execute the 'data_manipulation.py' file.

3.2 Model

For contrastive learning, a multilayer feedforward neural network was used. The network is designed to project an initial embedding vector into a smaller, normalized latent space. The network consists of four linear layers, with normalization, activation, and dropout applied to the first three layers. The activation function used is the Gaussian Error Linear Unit (GELU), which mitigates the sharp threshold introduced by ReLU. After the network, the produced vectors are normalized along the feature dimension (which is common for cosine similarity calculation).

3.3 Contrastive loss

The loss used is specific to contrastive learning. This loss function aims to bring the "positive" (similar) embedding vectors closer together and push the "negative" (dissimilar) vectors further apart.
Additionally, a 'margin' parameter was included in the loss to prevent the further "compression" of already very low similarities.
The formula for the loss function used is as follows.

$$\mathcal{L}(\mathbf{e}_1, \mathbf{e}_2, y) = y \cdot (1 - \cos\_\text{sim}(\mathbf{e}_1, \mathbf{e}_2))^2 + (1 - y) \cdot [\max(0, \cos\_\text{sim}(\mathbf{e}_1, \mathbf{e}_2) - m)]^2$$

If the two embeddings are supposed to be similar (label = 1), the goal is to bring the cosine similarity (cosine_sim) closer to 1. On the other hand, if the two embeddings are supposed to be dissimilar (label = 0), the objective is to ensure that cosine_sim $<$ margin. In fact, if cosine_sim $>$ margin, a penalty is applied, whereas if it is below the margin, the penalty is nullified. This design makes the margin a tolerance threshold for negative pairs.

3.4 Training

After performing hyperparameter tuning, such as adjusting the batch size, number of epochs, and threshold, training was conducted on the dataset of embedding pairs, which had been previously created and saved in binary format.
The training pipeline includes a testing phase immediately after the training is completed (since the network is lightweight, the training is not computationally intensive).

To replicate the training process, simply execute the 'build_model.py' file (ensuring that the steps from the previous sections have been followed).

4. Results

The testing phase involved comparing the tables in pairs (both positive and negative) and yielded the following results.

	Precision	**Recall**	**F1-score**
Pairs	0.53	087	0.66

Table 1: Performance metrics of solution

The results obtained are not very satisfactory, particularly in terms of Precision. The model tends to generate a significant number of false positives. However, in terms of Recall, the model is able to detect most of the positives, albeit at the cost of a higher number of false positives.

5. Conclusion

At this point, it is useful to make a few considerations. The experiment conducted has certainly provided an opportunity to explore common and powerful deep learning techniques, albeit in a simplified manner. However, the results obtained were not optimal (particularly in terms of Precision).
A possible improvement could be to use representations that do not involve embedding the entire table (attributes and the first 10 rows), but instead create representations based on individual columns (assigning weights depending on similarity).

Another alternative could be to extend the dataset or create pairs using different approaches, which might aid in generalization (in fact, by testing on the training set, slight overfitting can be observed).